Field Hours

CSCE 680 Testing and Diagnosis of Digital Systems

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Stuck-at, then collapse

You model a defect as a stuck-at fault on one line. Stuck-at-0 forces the line to 0 no matter what drives it. Stuck-at-1 forces it to 1. A 2-input AND has 6 single stuck-at faults if you count both inputs and the output, each in 2 polarities. You assume one fault at a time. A passing test does not call the chip perfect. It says those faults, alone, would have changed an output you observed.

You collapse equivalent faults before you count the work. On an AND, an input stuck-at-0 and the output stuck-at-0 both force the output to 0 for every input, so you keep 1 representative. Input stuck-at-1 is not the same as output stuck-at-1: the other input can still make the output 0. After the collapse this gate is 4 faults, not 6. You generate tests for the collapsed set and you remember what you merged.

Same gate, two polarities. Equivalent faults are one test, not two.

Stuck-at-0

Line forced to 0

AND input matches output

One class after collapse

Stuck-at-1

Line forced to 1

AND input is not the output

Keep it in the list

Six faults become four

1. A 2-input AND has faults on input a, input b, and the output, each stuck-at-0 and stuck-at-1. That is 6 faults.

2. Input a stuck-at-0, input b stuck-at-0, and output stuck-at-0 all force the output to 0. You keep 1 of them.

3. Input a stuck-at-1, input b stuck-at-1, and output stuck-at-1 stay distinct. The collapsed list has 4 faults, not 6.

Problem 1. How many single stuck-at faults does a 2-input AND have before collapse, and how many equivalence classes after the collapse in this lecture?

Answer. 6 before collapse: two inputs and the output, each stuck-at-0 and stuck-at-1. After collapse, 4.

The three stuck-at-0 faults are one class. The three stuck-at-1 faults are not equivalent to each other.

Problem 2. Why is an AND input stuck-at-1 not equivalent to the output stuck-at-1?

Answer. With the other input at 0, an input stuck-at-1 still yields output 0, while output stuck-at-1 yields 1. The two faults are distinguishable, so you keep both.

Lecture 2 · 90 minutes

Activate, justify, observe

You build a test by making the good circuit and the bad circuit disagree at a pin you can see. For an AND output stuck-at-0 you need the good output to be 1, so you set both inputs to 1. The faulty output is 0. That pair, good 1 and bad 0, is the discrepancy you move toward a primary output. If a later gate blocks it, the pattern does not detect the fault even though you activated it.

You justify side inputs so the discrepancy can pass. A 2-input OR passes it only if the other input is 0. Side input 1 makes the OR output 1 in both machines, and the fault is masked. An AND passes a discrepancy only if its side input is 1. With 2 side inputs, a contradiction sends you back. The pattern is the assignment that reaches a pin.

Activation without a path to a pin detects nothing. Side inputs are part of the pattern.

1. Set both AND inputs to 1

2. Good 1, faulty 0

3. Hold the OR side at 0

4. Read the pin

An AND feeding an OR

1. The fault is the AND output stuck-at-0. You set both AND inputs to 1. Good output is 1, faulty output is 0.

2. That discrepancy enters one input of a 2-input OR. You set the other OR input to 0 so the discrepancy reaches the OR output.

3. If you had set the side input to 1, both machines would drive the OR to 1 and the fault would be invisible. The pattern is the values that survive that check.

Problem 1. You want to detect an AND output stuck-at-0. What do you drive on the two inputs, and what do the good and bad outputs do?

Answer. Both inputs are 1. The good output is 1 and the faulty output is 0. If a later gate masks that pair, the pattern still does not detect the fault.

Problem 2. A discrepancy arrives on one input of a 2-input OR. What must the other input be, and what happens if you set it to 1?

Answer. The other input must be 0, or the discrepancy dies. If it is 1, both the good and the bad machine produce 1, and the fault is masked.

Lecture 3 · 90 minutes

Coverage after fault dropping

You grade patterns by fault simulation. You simulate the good machine once, then each faulty machine, and you mark a fault detected when some output differs. With 50 collapsed faults and 45 detected, coverage is 45/50 = 0.90. The other 5 are undetected. A long set that still misses them is not finished until you know whether they are redundant or merely untested.

You drop a fault once some pattern detects it. After early patterns, 45 of 50 are gone and the next pattern is scored against the remaining 5, so later patterns simulate less. A fault that no pattern can detect may be redundant: nothing makes good and bad differ. Pushing coverage to 100 percent fails if 2 of the 50 are redundant. You separate those 2 from faults you have not hit yet.

Drop detected faults. The last few simulations are about the residue, not the whole list.

Simulate the good machine -> Simulate remaining faults -> Mark a differing pin -> Drop the detected ones

Forty-five of fifty

1. The collapsed list has 50 faults. Patterns so far make 45 of them disagree with the good machine on some pin.

2. Coverage is 45/50 = 0.90. You drop those 45. The next pattern is simulated against 5 faults, not 50.

3. If 2 of those 5 can never be detected, they are redundant. The other 3 are still your test-generation job. Reporting 100 percent would be false.

Problem 1. Patterns detect 45 of 50 collapsed faults. What is the coverage, and how many faults does the next pattern need to be simulated against if you drop detected faults?

Answer. Coverage is 45/50 = 0.90. The next pattern is simulated against the remaining 5, not against all 50.

Problem 2. Two of the 50 faults are redundant. Why will more patterns never reach coverage 1?

Answer. No input makes the good machine and that faulty machine differ, so no pattern detects them. You separate those 2 from the faults you have not tried. The ceiling is 48/50 = 0.96 if the other 48 are detectable.

Lecture 4 · 90 minutes

A scan chain

You cannot set internal flip-flops from the pins the way you set the AND inputs. Scan adds a second path. Each flip-flop gets a multiplexer on its data input. In scan mode the flops form one shift register. You shift a pattern in, apply 1 functional clock, and shift the captured bits out. A chain of 100 flops needs 100 scan clocks to load and 100 to unload, plus that 1 capture, for every pattern.

You pay area and time for scan. The multiplexer adds delay, and scan enable costs a pin unless you reuse one. For 20 patterns on a chain of 100, without overlapping the two shifts, clocks are 20 times (200 + 1) = 4020. You

turn scan off in mission mode, or the chip is only a shift register. Scan lets pattern generation treat the cloud as combinational logic.

Mission mode uses the functional input. Scan mode chains the flops into one shift register.

Combinational cloud

Mux on each flip-flop

Chain of 100

One capture clock

Twenty patterns, chain of 100

1. Each pattern shifts 100 bits in, pulses 1 capture clock, and shifts 100 bits out. That is 201 clocks, and you are not overlapping the shifts.

2. Twenty patterns cost 20 times 201 = 4020 clocks.

3. In mission mode scan enable is off, so the multiplexer selects functional data. Left on, every flop is only a bit in a shift register.

Problem 1. A chain has 100 flops and you do not overlap shift-in with shift-out. How many clocks does 1 pattern take, and how many do 20 patterns take?

Answer. One pattern takes 100 + 100 + 1 = 201 clocks. Twenty patterns take 20 times 201 = 4020 clocks.

Problem 2. Why must scan enable be off in mission mode?

Answer. If it stays on, the flops form a shift register and the chip is not the functional design. The multiplexer must select the functional data input. Scan is a test mode, not the product mode.

Lecture 5 · 90 minutes

Patterns from a register

You generate patterns on the chip when a tester cannot feed a long chain at speed. A linear feedback shift register makes them. A 4-bit register with the right taps visits 15 nonzero states, because 2 to the fourth minus 1 is 15, then it repeats. The all-zero state sticks at 0, so you do not seed 0. Each clock is a new pattern for the combinational block. You did not store those 15 patterns in a table.

You compact responses so you do not shift every result off the chip. A signature register folds each response into a running code of 16 bits, and you compare that 1 code with the good signature. Two streams can share a code. There are 65536 codes, so a random bad stream hits the good one about 1 time in 65536. You accept that or you add bits.

Do not seed the all-zero state. Compact the responses to one code and accept a tiny alias risk.

1. Seed a nonzero 4-bit state

2. Clock through 15 patterns

3. Fold responses to 16 bits

4. Compare one signature

Fifteen patterns, one signature

1. A 4-bit register with the right taps has 15 nonzero states. You seed anything but 0, or it stays at 0.

2. Fifteen clocks produce 15 patterns. You do not store them off-chip.

3. Responses fold into a 16-bit signature. A wrong stream matches the good signature about 1 time in 65536. You compare that one code, not every response bit.

Problem 1. What is the period of a maximal 4-bit linear feedback shift register, and why do you avoid the seed 0?

Answer. 15, which is 2 to the fourth minus 1. The all-zero state has period 1: it sticks at 0 and never produces the other patterns.

Problem 2. A 16-bit signature has how many codes, and about how often does a random bad stream match the good signature?

Answer. 65536 codes. A random bad stream matches about 1 time in 65536. That alias is why a passing signature is not a proof that every response bit was correct.

Lecture 6 · 90 minutes

Marching through a memory

You test a memory as cells, not as a cloud of gates. A march walks addresses in one direction and applies a fixed list of reads and writes. For 4 addresses, an up march touches 0, then 1, then 2, then 3. A down march touches 3, then 2, then 1, then 0. The element read-0 write-1 expects 0 and leaves 1. You fail the chip on the first read that returns the wrong bit.

You pick the march for the fault you claim. Stuck-at cells fall to a short march: write 0 everywhere, then upward read 0 and write 1, then downward read 1 and write 0. On 4 cells that is 4 writes, plus 8 operations upward, plus 8 downward, so 20 operations. A coupling fault, where a write to cell i flips cell j, can hide on an up-only walk. You keep the down pass, and you record the 20.

Direction is part of the test. Up alone can miss a coupling that down exposes.

Write 0 on the way up

Read 0, write 1, upward

Read 1, write 0, downward

20 operations on 4 cells

Twenty operations on four cells

1. Upward, write 0 at addresses 0, 1, 2, and 3. That is 4 writes.

2. Upward again, read 0 and write 1 at each address: 8 operations. Downward, read 1 and write 0 at

each address: 8 more.

3. The total is 4 + 8 + 8 = 20. The first wrong read fails the chip. Dropping the downward pass can hide a coupling that only shows up when addresses fall.

Problem 1. The march is write 0 up, then read 0 write 1 up, then read 1 write 0 down, on 4 cells. How many operations is that?

Answer. 20. The first element is 4 writes. Each of the other two elements is 4 reads and 4 writes, so 8 plus 8. 4 + 8 + 8 = 20.

Problem 2. Why keep the downward pass if the upward pass already wrote and read every cell?

Answer. A coupling fault can depend on direction: a write to a higher address flips a lower cell, or the reverse. An up-only walk can miss it. You record the down pass as part of the 20 operations.

Lecture 7 · 90 minutes

Intersect the suspects

You diagnose by intersecting the faults that predict the failures. Pattern P1 fails, and only faults f2 and f5 in the dictionary predict that fail. Pattern P2 fails, and only f5 and f7 predict it. The intersection is f5, so you indict f5. Stop after P1 and you still have 2 suspects. The second pattern is what isolates 1. The dictionary stores those predictions so the bench does not resimulate during the session.

You sometimes get an empty set or a pair. Empty means the defect is not a single stuck-at in the dictionary, or the setup is wrong. A pair no pattern separates is indistinguishable, as with equivalent faults on the AND. You cannot name 1 net when both explain all 3 failing patterns and the passes. You report the set, not a guess from the first red pin.

Each failing pattern contributes a set. The diagnosis is the intersection, which can be empty.

After pattern P1

Suspects f2 and f5

Two explanations

Do not stop here

After pattern P2

Suspects f5 and f7

Intersection is f5

One site indicted

Two patterns, one site

1. P1 fails. The dictionary says only f2 and f5 would cause that failure. You have 2 suspects.

2. P2 fails. The dictionary says only f5 and f7 would cause it.

3. The intersection is f5. You indict f5, not f2 and not f7. If the intersection had been empty, the single

stuck-at model would not explain this chip.

Problem 1. P1 is explained only by f2 and f5. P2 is explained only by f5 and f7. Which fault do you indict?

Answer. f5. It is the only fault in both sets. Stopping after P1 would leave 2 suspects. The dictionary is what lists those predictions.

Problem 2. The intersection of the explaining sets is empty after 3 failing patterns. What do you conclude?

Answer. The defect is not a single stuck-at from the dictionary, or the setup is wrong. You do not pick a favorite net from the first red pin. An indistinguishable pair would be reported as a set of 2, not as one name.

Lecture 8 · 90 minutes

A test plan with two ledgers

You can now take a small netlist from a fault list to a scan plan. You list stuck-at sites, collapse an AND from 6 faults to 4, and sensitize a path so good and bad differ at a pin. You grade coverage as 45 out of 50. You insert a scan chain of 100 flops when state is not controllable, and you can replace patterns with a register of period 15.

You can also read a failing chip. A march of 20 operations on 4 cells catches the cell faults you built it for, and intersecting 2 failing patterns can cut 2 suspects to 1. Coverage and diagnosis answer different questions. A figure of 0.90 says how much of the list you can light up. The intersection says which site explains this chip. You can write both numbers and not mix them.

Coverage grades the list. Diagnosis names a site on one chip. Do not swap the sentences.

1. Collapse, then sensitize

2. Grade 45 of 50

3. Time the chain of 100

4. Intersect failing patterns

Coverage and scan time

1. The list has 50 collapsed faults and 45 are detected. Coverage is 0.90. The other 5 are not cured by calling the scan chain longer.

2. The chain has 100 flops and you apply 20 patterns with no shift overlap. Each pattern is 201 clocks, and 20 patterns are 4020 clocks.

3. A separate chip fails 2 patterns whose suspect sets intersect in 1 fault. That name is a diagnosis. It is not a new coverage number.

Problem 1. You have 50 collapsed faults and the patterns detect 45. A chain of 100 flops runs 20 patterns with no shift overlap, at 2 full shifts plus 1 capture per pattern. What is the coverage, and how many clocks do you spend?

Answer. Coverage is 45/50 = 0.9. Clocks are 20 times (2 times 100 plus 1) = 20 times 201 = 4020. More scan clocks do not detect the other 5 unless new patterns actually detect them. Coverage and scan time are different ledgers.

Problem 2. A march on 4 cells uses 20 operations, and two failing patterns have suspect sets {f2, f5} and {f5, f7}. What does the march count, and which fault do you indict?

Answer. The 20 counts reads and writes of the march, not stuck-at coverage. The intersection of the two sets is f5, so you indict f5. The march and the dictionary answer different questions about the same chip.